MODELS AND ROBOTICS SECTION

MARS

Open Projects 2026

✦ ✦ ✦

ARTIFICIAL INTELLIGENCE

MACHINE LEARNING

WEB DEVELOPMENT

Problem Statements & Deliverables

PART I

Artificial Intelligence & Machine Learning

Problem Statements

01

PROBLEM STATEMENT 1

Support Integrity Auditor (SIA)

Artificial Intelligence / Machine Learning

NLP

CRM Systems

§ 1 BACKGROUND

In enterprise-scale CRM ecosystems, manual ticket triage is riddled with agent fatigue bias, customer favoritism, and keyword anchoring. When critical issues are mislabeled as "Low" or trivial complaints are inflated to "Critical," Service Level Agreements (SLAs) are jeopardized, and customer churn increases. Existing rule-based or keyword-matching systems fail to detect the nuanced discrepancies between a ticket's true severity and its assigned priority.

This project addresses a fundamentally harder variant of the triage problem: there are no pre-annotated mismatch labels. The system must bootstrap its own supervision signal from raw ticket data alone.

§ 2 PROBLEM STATEMENT

Build the Support Integrity Auditor (SIA) — a semantics-driven, evidence-grounded automated auditor that detects *Priority Mismatch*: cases where the objective characteristics of a support ticket (text, customer domain, channel, resolution time) conflict with its human-assigned priority level.

THE SYSTEM MUST:

- Infer the "true" severity of a ticket independent of its assigned label.
- Generate its own binary mismatch supervision signal (self-supervised).
- Train a fine-tuned classifier on that pseudo-labeled data.
- Produce a structured, hallucination-free Evidence Dossier for every flagged ticket.
- Generalize to previously unseen tickets, including adversarially crafted examples.

§ 3 DATASET

Recommended Dataset: *Customer Support Tickets — CRM Dataset*

Dataset Link: kaggle.com/datasets/ajverse/customer-support-tickets-crm-dataset/data

KEY COLUMNS USED

COLUMN	ROLE
Ticket Subject	Short summary of the issue
Ticket Description	Full natural language problem statement
Customer Email / Product Purchased	Proxy for customer tier / domain
Ticket Priority	Human-assigned label (Low / Medium / High / Critical)
Ticket Channel	Intake channel (email, chat, phone, social media)
Resolution Time	Time to resolve — indirect severity signal
Ticket Type	Category of issue

§ 4 MANDATORY PIPELINE STAGES

STAGE 1: PSEUDO-LABEL GENERATION (SELF-SUPERVISED)

Construct a reproducible pipeline that assigns each ticket an inferred severity, independent of the Ticket Priority column. A binary mismatch label is then derived by comparing the inferred severity to the

assigned priority. The pipeline must fuse at least two of the following independent signals:

- **LLM-based zero-shot severity scoring:** Using an open-source model, e.g., Mistral-7B-Instruct or Phi-3-mini.
- **Embedding-based clustering:** e.g., sentence-transformers, for semantic urgency grouping.
- **Resolution-time regression:** Used as a severity proxy.
- **Rule-based NLP features:** Keyword density, negation detection, escalation phrases.

> *Requirement: Justify your fusion strategy in the README with an ablation showing each signal's individual contribution.*

STAGE 2: CLASSIFIER TRAINING (FINE-TUNED MODEL)

Train a supervised binary classifier on the pseudo-labeled data.

- **Model:** Must use a fine-tuned or adapter-trained model (e.g., LoRA on a small LLM, or fine-tuned DeBERTa-v3-small) — not a frozen zero-shot pipeline.
- **Inputs:** Input features must include text fields and at least one structured metadata feature (channel, domain tier, or resolution time).
- **Imbalance:** Class imbalance must be explicitly addressed (e.g., weighted loss, SMOTE, or oversampling).

STAGE 3: EVIDENCE DOSSIER GENERATION

For every ticket classified as a mismatch, generate a structured dossier using the exact schema below:

SCHEMA

```
{
  "ticket_id": "...",
  "assigned_priority": "...",
  "inferred_severity": "...",
  "mismatch_type": "Hidden Crisis | False Alarm",
  "severity_delta": "",
  "feature_evidence": [
    { "signal": "keyword", "value": "...", "weight": "..." },
    { "signal": "resolution_time", "value": "...", "interpretation": "..." }
  ],
  "constraint_analysis": "<2-3 sentence grounded explanation>",
  "confidence": ""
}
```

> *Hard Rule: Every feature_evidence item must be traceable to a specific field in the input ticket. Any fabricated or unverifiable claim (hallucination) results in immediate disqualification of that test case.*

---

§ 5 EVALUATION METRICS

Models will be evaluated using the following metrics:

- Binary Classification Accuracy (%)
- Macro F1 Score
- Per-Class Recall (both Consistent and Mismatched classes)
- Pseudo-Label Signal Agreement (pairwise agreement between the two chosen signals)

— Dossier Quality (feature grounding, zero hallucination, severity delta calibration, mismatch type accuracy)

Secondary / Bonus: Adversarial robustness test on 10 held-out tickets designed to fool keyword-based systems. Systems scoring ≥ 7/10 receive a 10% score bonus.

§ 6 VERIFICATION CRITERIA

A submission is considered valid and successfully completed only if all three of the following conditions are satisfied on the held-out evaluation split:

METRIC	MINIMUM THRESHOLD
Binary Classification Accuracy	**≥ 83%**
Macro F1 Score	**≥ 0.82**
Per-Class Recall	**≥ 0.78 (on both classes)**

Submissions failing to meet any single threshold will not be considered verified. Dossiers containing hallucinated evidence will result in disqualification of the affected test cases regardless of classification scores.

§ 7 EXPECTED DELIVERABLES

1. GITHUB REPOSITORY

- `notebook.ipynb` — Full reproducible pipeline (pseudo-labeling → training → inference).
- `train_pipeline.py` — Standalone training script.
- `predict.py` — Inference script accepting a CSV and outputting predictions + dossiers.
- `README.md` — Methodology, architecture diagram, ablation table, and metric results.
- `requirements.txt` — Pinned dependencies.

2. STREAMLIT WEB APP (HOSTED URL)

- Accepts single-ticket form input or batch CSV upload.
- Returns binary judgment + full Evidence Dossier per ticket.
- Displays a Priority Mismatch Dashboard with distribution of flagged tickets, mismatch types, and top contributing signals.
- Includes a severity delta heatmap across ticket categories and channels.

3. DEMO VIDEO (~3 MINUTES)

- Walkthrough of one "Hidden Crisis" and one "False Alarm".
- Explanation of the pseudo-label generation strategy.
- Live adversarial ticket input demonstration.

02

PROBLEM STATEMENT 2

Deepfake Audio Detection

Machine Learning | Deep Learning | Audio Processing

§ 1 BACKGROUND

Advances in generative AI have enabled the creation of highly realistic synthetic speech, commonly known as deepfake audio. Such audio can be misused for impersonation, fraud, misinformation, and social engineering attacks. Detecting whether an audio

recording is genuine or artificially generated is therefore an important research and security challenge.

§ 2 PROBLEM STATEMENT

Develop a machine learning or deep learning system capable of classifying speech recordings as either *Genuine (Human)* or *Deepfake (AI-Generated)*. Participants may employ feature-based, end-to-end, or hybrid approaches. The final model should generalize well to previously unseen audio samples and demonstrate robust performance on the evaluation dataset.

§ 3 DATASET

Recommended Dataset: *The Fake-or-Real Dataset*
Dataset Link: kaggle.com/datasets/mohammedabdeldayem/the-fake-or-real-dataset

Use the *train* folder in the *LA norm* directory for training. Additional reference benchmark: ASVspoof 2019 dataset (asvspoof.org/index2019.html) may be used for supplementary training or cross-dataset generalization testing.

§ 4 EVALUATION CRITERIA

PRIMARY METRICS

— **Overall Accuracy ≥ 80%** — Percentage of audio samples correctly classified across both classes on the evaluation set.

— **Equal Error Rate (EER) ≤ 12%** — The point where the false acceptance rate equals the false rejection rate. A lower EER indicates a more balanced and reliable detector.

SECONDARY METRICS

— **Confusion Matrix** — Must be reported. Reveals per-class performance and error distribution between Genuine and Deepfake predictions.

— **F1 Score ≥ 80%** — Harmonic mean of precision and recall. Ensures the model does not exploit class imbalance.

— **Per-Class Accuracy ≥ 75%** — Both Genuine and Deepfake classes must individually exceed 75% accuracy.

§ 5 VERIFICATION CRITERIA

A submission will be considered valid and successfully completed only if *both* of the following conditions are satisfied on the evaluation dataset:

METRIC	REQUIRED THRESHOLD
Overall Accuracy	**≥ 80%**
Equal Error Rate (EER)	**≤ 12%**

Submissions failing to meet either threshold will not be considered verified. Accuracy will additionally be checked on a custom private test dataset that will not be shared publicly.

§ 6 EXPECTED DELIVERABLES

GITHUB REPOSITORY

- ipynb notebook with full running code.
- Trained model.
- Python script where the model can be tested by feeding new audio samples.
- Performance report including accuracy, EER, F1 score, and confusion matrix.
- Description of preprocessing, feature extraction, and model architecture.
- Clear and detailed `README.md` including project description, methodology, pipeline, and metrics.

STREAMLIT WEB APP (HOSTED)

- Receives an audio file as input.
- Returns whether it is Genuine (Human) or Deepfake (AI-Generated), along with the confidence score.

DEMO VIDEO (~2 MINUTES)

- Screen recording demonstrating the use-case of the web app.

PART II

Web Development

Problem Statements

01

PROBLEM STATEMENT 1

Unified Campus Intelligence Dashboard with AI Assistant

React.js / Next.js | MCP Servers | LLM Integration

§ 1 THE PROBLEM

College campuses have data scattered everywhere: the library uses one legacy portal, the cafeteria menu is a PDF on a website, club events are on Google Calendars, and academic handbooks are massive PDFs. Students waste time digging through 5 different systems just to find out if a book is available or what time a tech fest workshop starts.

§ 2 THE SOLUTION

Build a Unified Web Dashboard featuring an embedded AI Assistant. Instead of building massive, brittle web scrapers that dump everything into one giant database, students will build independent MCP (Model Context Protocol) Servers for each campus data source. The AI will dynamically query these servers in real-time based on what the student asks.

§ 3 KEY FEATURES

- Independent MCP Servers for distinct campus data sources (library, cafeteria, events, academics, etc.).
- AI Assistant that routes natural-language queries to the appropriate MCP server(s) in real-time.
- Unified dashboard UI that surfaces results from multiple sources in one view.
- No single giant database — data is fetched live from each source server.
- (Optional) Authentication / personalization for students.

§ 4 TECH STACK SUGGESTIONS

LAYER	SUGGESTED TECHNOLOGIES
Frontend	React.js or Next.js
Backend / MCP Servers	Node.js or Python (FastAPI / Flask)
AI Integration	Any LLM API with tool/function-calling support
Hosting	Vercel / Netlify (frontend), Render / Railway (backend)

§ 5 SUBMISSION INSTRUCTIONS

1. GITHUB REPOSITORY

- Push the complete project code to a public GitHub repo.
- Include a clear and detailed `README.md` with project description, features, tech stack, setup instructions, and deployed demo link.

2. DEMO VIDEO (5–10 MINUTES)

- Upload a screen-recorded video demonstrating all core features working live.

- Upload to Google Drive or YouTube and share the link.

02

PROBLEM STATEMENT 2

P2P Web Share — Direct Browser-to-Browser File Transfer

WebRTC | Node.js | React.js

§ 1 OBJECTIVE

Traditional file-sharing services rely on central servers, resulting in heavy bandwidth costs and storage limitations. The objective of this project is to build a lightweight, decentralized P2P file sharing web application. Using this platform, users can drop a file to generate a unique share room link. Recipients who open this link connect directly to the sender's browser to stream the file. A lightweight central signaling server coordinates the initial handshake but never reads, processes, or stores any part of the file data.

§ 2 KEY FEATURES — CORE MVP

For a successful basic submission, the application must achieve stable, secure 1-to-1 direct transfers:

- **Share Room Creation:** Drag-and-drop zone to upload files (limit <50MB for standard browser memory) and generate a unique Room ID or invite link.
- **Signaling Handshake:** A simple Node.js / Socket.io signaling backend to coordinate initial WebRTC connection offers and answers.
- **Direct P2P Transfer:** Once connected, read the file using the browser FileReader API and transfer it directly to the receiver over WebRTC data channels.
- **Basic Chunk Verification:** Generate and verify a cryptographic hash (e.g., SHA-256) of the file chunks before and after transfer to guarantee zero data corruption.
- **Progress Indicators & Connection Status:** A real-time UI showing transfer percentage, transfer speed (MB/s), and active connection status.
- **Graceful Disconnect Handling:** Ensure the application does not crash or freeze if a user closes a tab or disconnects. The UI should gracefully notify the remaining user of the connection drop.
- **Auto-Download:** Reassemble the incoming verified chunks in the receiver memory and automatically trigger a local file download when completed.

§ 3 BROWNIE POINTS / ADVANCED FEATURE EXTENSIONS (OPTIONAL)

Implement any of these advanced extensions to make your project stand out:

- **Multi-Peer Support (Mesh Swarming):** Allow a third peer joining the room to download different parts of the file from both the sender and the second peer simultaneously.
- **Large File Support (>500MB):** Bypass standard browser RAM limitations by writing incoming chunks directly to local storage using the Streams API paired with Origin Private File System (OPFS) or IndexedDB.
- **Zero-Knowledge Encryption:** Encrypt the file chunks in the browser using the Web Crypto API (AES-GCM) before transmitting. Pass the decryption key purely via the URL hash (e.g., /#key=...) so the signaling server never has access to the raw file data.
- **Connection Churn Recovery (Auto-Resume):** Implement a state-tracking handshake so that if a connection drops mid-transfer, the download can automatically pause and resume from the last verified chunk once the peer reconnects, rather than restarting from 0%.

§ 4 TECH STACK SUGGESTIONS

LAYER	SUGGESTED TECHNOLOGIES
Frontend	React.js, Tailwind CSS
P2P Communication	WebRTC API, or wrapper libraries like PeerJS or Simple-Peer
Backend Signaling	Node.js + Express.js + Socket.io
Hosting	Vercel / Netlify (Frontend), Render / Railway (Backend)

§ 5 SUBMISSION INSTRUCTIONS

1. GITHUB REPOSITORY

- Public repo containing clean, commented frontend and backend code.
- Include a `README.md` with project description, setup instructions, and deployment links.

2. DEMO VIDEO (~3 MINUTES)

- A short screen recording demonstrating a live file transfer between two different browser windows or devices.

- Upload to Google Drive or YouTube and share the link.

— NOTICE —

Submissions found to be copied from GitHub or other teams will be disqualified. All work must be original and done by you / your team. Feel free to use open-source tools and libraries, but ensure your implementation and logic are entirely your own. Submissions utilising copy-pasted baseline WebRTC templates without customised UI, active progress indicators, and proper error feedback will be disqualified.